PIZZA SALES DATA ANALYSIS

Project by,

Nisarga P

Data Analyst

Project Overview:

- The dataset contains pizza sales data including orders, order details, pizza types, and pricing information
- Tables used: Orders, Order_Details, Pizzas, Pizza_Types
- The analysis focuses on:

✓ Sales performance

✓ Customer behavior

✓ Product popularity

✓ Revenue contribution

✓ Time-based demand patterns

Objective:

To analyze pizza sales data using SQL in order to uncover key business insights such as,

- Revenue trends
- Customer ordering behavior
- Popular products
- Category performance
- Hourly order demands

To support data-driven decision-making.

1. Retrieve the total number of orders placed.

```
select count(order_id) as total_orders
from orders;
```

Result Grid

	total_orders
▶	21350

Insights: The total number of orders reflects overall business activity and demand volume.

2. Calculate the total revenue generated from pizza sales.

```
select round(sum(o.quantity * p.price),2) as total_sales
from order_details as o
join pizzas as p
on p.pizza_id = o.pizza_id;
```

Result Grid

	total_sales
▶	817860.05

Insights: Total revenue indicates the overall financial performance of the business.

3. Identify the highest priced pizza.

```
select pizza_types.name, pizzas.price
from pizza_types
join pizzas
on pizzas.pizza_type_id = pizza_types.pizza_type_id
order by pizzas.price desc
limit 1
;
```

Result Grid | Filter Rows:

name	price
The Greek Pizza	35.95

Insights: Premium-priced pizzas contribute higher revenue per order and target high-value customers.

4. Identify the most common pizza size ordered.

```
select pizzas.size, count(order_details.order_details_id) as order_count
from pizzas
join order_details
on order_details.pizza_id = pizzas.pizza_id
group by pizzas.size
order by order_count desc;
```

Result Grid | Filter

size	order_count
L	18526
M	15385
S	14137
XL	544
XXL	28

Insights: The most frequently ordered size shows customer preference and helps in inventory planning.

5. List the top 5 most ordered pizza types along with their quantities.

```
select pizza_types.name, sum(order_details.quantity) as quantity
from pizza_types
join pizzas
on pizza_types.pizza_type_id = pizzas.pizza_type_id
join order_details
on order_details.pizza_id = pizzas.pizza_id
group by pizza_types.name
order by quantity desc
limit 5;
```

Result Grid | Filter Rows:

name	quantity
The Classic Deluxe Pizza	2453
The Barbecue Chicken Pizza	2432
The Hawaiian Pizza	2422
The Pepperoni Pizza	2418
The Thai Chicken Pizza	2371

Insights: A small number of pizza types drive the majority of sales, indicating popular customer choices.

6. Join the necessary tables to find the total quantity of each pizza category ordered.

```
select pizza_types.category, sum(order_details.quantity) as quantity
from pizza_types
join pizzas
on pizza_types.pizza_type_id = pizzas.pizza_type_id
join order_details
on pizzas.pizza_id = order_details.pizza_id
group by pizza_types.category
order by quantity desc;
```

Result Grid | Filter

category	quantity
Classic	14888
Supreme	11987
Veggie	11649
Chicken	11050

Insights: Certain pizza categories dominate sales, helping identify high-performing product segments.

7. Determine the distribution of orders by hour of the day.

```
select hour(order_time) as hours, count(order_id)
from orders
group by hours;
```

Result Grid | Filter Rows

hours	count(order_id)
11	1231
12	2520
13	2455
14	1472
15	1468
16	1920
17	2336
18	2399
19	2009
20	1642
21	1198
22	663
23	28
10	8
9	1

Insights: Order distribution highlights peak hours, which are crucial for staffing and operations planning.

8. Join relevent tables to find the category wise distribution of pizzas.

```
select category, count(name)
from pizza_types
group by category;
```

Result Grid | Filter Row

	category	count(name)
▶	Chicken	6
	Classic	8
	Supreme	9
	Veggie	9

Insights: The menu distribution shows how product variety is spread across categories.

9. Group the orders by date and calculate the average number of pizzas ordered per day.

```
select round(avg(quantity),0) as avg_pizzas_per_day
from(
  select orders.order_date, sum(order_details.quantity) as quantity
  from orders
  join order_details
  on orders.order_id = order_details.order_id
  group by orders.order_date) as order_quantity;
```

Result Grid

	avg_pizzas_per_day
▶	138

Insights: Average daily orders help in understanding demand consistency and forecasting.

10.Determine the top 3 most ordered pizza types based on revenue.

```
select pizza_types.name, sum(order_details.quantity * pizzas.price) as revenue
from pizza_types
join pizzas
on pizza_types.pizza_type_id = pizzas.pizza_type_id
join order_details
on pizzas.pizza_id = order_details.pizza_id
group by pizza_types.name
order by revenue desc
limit 3;
```

Result Grid | Filter Rows:

name	revenue
The Thai Chicken Pizza	43434.25
The Barbecue Chicken Pizza	42768
The California Chicken Pizza	41409.5

Insights: High-revenue pizzas are key drivers of profitability, even if not the most ordered.

11.Calculate the percentage contribution of each pizza type to total revenue.

```
select pizza_types.category, round(sum(order_details.quantity * pizzas.price) /
  (select round(sum(order_details.quantity * pizzas.price),2)  as total_revenue
    from order_details
    join pizzas
    on order_details.pizza_id = pizzas.pizza_id) *100,0) as revenue
from pizza_types
join pizzas
on pizza_types.pizza_type_id = pizzas.pizza_type_id
join order_details
on order_details.pizza_id = pizzas.pizza_id
group by pizza_types.category
order by revenue desc;
```

Result Grid | Filter

category	revenue
Classic	27
Supreme	25
Veggie	24
Chicken	24

Insights: Revenue contribution highlights which categories generate the most income for the business.

12. Analyze the cumulative revenue generated over time.

```
select order_date,
  round(sum(revenue) over(order by order_date),0) as cum_revenue
from
  (select orders.order_date, sum(order_details.quantity * pizzas.price) as revenue
  from orders
  join order_details
  on orders.order_id = order_details.order_id
  join pizzas
  on pizzas.pizza_id = order_details.pizza_id
  group by orders.order_date) as sales;
```

Result Grid | Filter Rows:

order_date	cum_revenue
2015-01-01	2714
2015-01-02	5446
2015-01-03	8108
2015-01-04	9864
2015-01-05	11930
2015-01-06	14358
2015-01-07	16561
2015-01-08	19399
2015-01-09	21526
2015-01-10	23990
2015-01-11	25863
2015-01-12	27782
2015-01-13	29831
2015-01-14	32359
2015-01-15	34344
2015-01-16	36938
2015-01-17	39002
2015-01-18	40979
2015-01-19	43366
2015-01-20	45764
2015-01-21	47804
2015-01-22	50301
2015-01-23	52725
2015-01-24	55014

Result Grid | Filter Rows

order_date	cum_revenue
2015-01-24	55014
2015-01-25	56631
2015-01-26	58516
2015-01-27	61044
2015-01-28	63060
2015-01-29	65105
2015-01-30	67375
2015-01-31	69793
2015-02-01	72983
2015-02-02	75311
2015-02-03	77926
2015-02-04	80160
2015-02-05	82376
2015-02-06	84886
2015-02-07	87123
2015-02-08	89158
2015-02-09	91354
2015-02-10	93410
2015-02-11	95870
2015-02-12	98029
2015-02-13	100783
2015-02-14	103103
2015-02-15	105244
2015-02-16	107213

Result Grid | Filter Rows

order_date	cum_revenue
2015-02-16	107213
2015-02-17	109334
2015-02-18	111977
2015-02-19	114008
2015-02-20	116899
2015-02-21	119010
2015-02-22	120590
2015-02-23	122758
2015-02-24	124953
2015-02-25	127294
2015-02-26	129555
2015-02-27	132413
2015-02-28	134953
2015-03-01	136551
2015-03-02	138930
2015-03-03	141218
2015-03-04	143663
2015-03-05	146013
2015-03-06	148527
2015-03-07	150928
2015-03-08	153116
2015-03-09	155450
2015-03-10	157839
2015-03-11	160047

Insights: Cumulative revenue shows growth trends and helps track business performance over time.

13.Determine the top 3 most ordered pizza types based on revenue for each pizza category.

```
select name, revenue, rn
from
  (select category, name, revenue, rank() over (partition by category order by revenue desc) as rn
  from
    (select pizza_types.category, pizza_types.name, round(sum(order_details.quantity * pizzas.price),0) as revenue
     from pizza_types
     join pizzas
     on pizza_types.pizza_type_id = pizzas.pizza_type_id
     join order_details
     on order_details.pizza_id = pizzas.pizza_id
     group by pizza_types.category, pizza_types.name) as a) as b
where rn<=3;
```

Result Grid | Filter Rows:

name	revenue	rn
The Thai Chicken Pizza	43434	1
The Barbecue Chicken Pizza	42768	2
The California Chicken Pizza	41410	3
The Classic Deluxe Pizza	38180	1
The Hawaiian Pizza	32273	2
The Pepperoni Pizza	30162	3
The Spicy Italian Pizza	34831	1
The Italian Supreme Pizza	33477	2
The Sicilian Pizza	30940	3
The Four Cheese Pizza	32266	1
The Mexicana Pizza	26781	2
The Five Cheese Pizza	26066	3

Insights: Identifies the best-performing pizzas within each category, enabling targeted promotions.

Key Insights:

- Certain categories dominate both quantity and revenue
- A limited number of pizza types contribute significantly to total sales
- Customer demand varies by time, with clear peak ordering hours
- Premium pizzas contribute more revenue per order
- Daily average orders indicate stable demand patterns
- Revenue trends show consistent business growth over time.

Skills Demonstrated:

- SQL Joins
- Aggregations
- Subqueries
- Windows Functions
- Data Analysis

Conclusion:

- This project strengthened my ability to analyze data using SQL and derive actionable insights.
- It highlights customer preferences, revenue drivers, and operational patterns, helping businesses make informed decisions.